

@notifier

Appears in: **DL-08**

server/notifier.py:17-51, 86-205

"All the sending logic lives in @notifier." ... "The notifier reads its settings from the config at the moment it is called, not when it is first loaded."

@notifier

server/notifier.py:17-51

```
17 def _desktop(title: str, message: str) -> None:
18     if not config.NOTIFY_DESKTOP:
19         return
20     try:
21         from plyer import notification
22         notification.notify(
23             title=title,
24             message=message[:256],
25             app_name="Betsy AI",
26             timeout=10,
27         )
28     except Exception:
29         pass
30
31
32 def _email(subject: str, html: str) -> None:
33     if not (config.NOTIFY_EMAIL and config.SMTP_USER and config.SMTP_HOST):
34         return
35     threading.Thread(target=_email_worker, args=(subject, html), daemon=True).start()
36
37
38 def _email_worker(subject: str, html: str) -> None:
39     try:
40         msg = MIMEMultipart("alternative")
41         msg["Subject"] = f"[Betsy] {subject}"
42         msg["From"] = config.SMTP_USER
43         msg["To"] = config.NOTIFY_EMAIL
44         msg.attach(MIMEText(html, "html"))
45         with smtplib.SMTP(config.SMTP_HOST, config.SMTP_PORT, timeout=10) as srv:
46             srv.ehlo()
47             srv.starttls()
48             srv.login(config.SMTP_USER, config.SMTP_PASS)
49             srv.sendmail(config.SMTP_USER, [config.NOTIFY_EMAIL], msg.as_string())
50     except Exception:
51         pass
```

...

server/notifier.py:86-205

```
86 def notify_approval_required(item: dict) -> None:
87     """Fire when any decision is queued for human review."""
88     if not config.NOTIFY_ON_APPROVAL:
89         return
90
91     sku = item.get("sku_id", "Unknown SKU")
92     supplier = item.get("supplier_id", "Unknown supplier")
93     total = item.get("po_total") or 0
94     action = item.get("action", "review")
95     reason = (item.get("reasoning") or "")[:160]
96
97     action_label = {
98         "generate_po": "Purchase order",
99         "flag_duplicate": "Duplicate invoice flagged",
100         "flag_for_approval": "Price spike",
101         "escalate": "Supplier unavailable",
102     }.get(action, action.replace("_", " ").title())
103
104     desktop_msg = (
105         f"{action_label}: {sku}\n"
106         f"{'Supplier: {supplier} | ' if supplier else ''}"
107         f"{'Total: €{total:,.0f}' if total else ''}\n"
108         f"{reason}"
109     )
110     _desktop(f"Betsy needs your OK – {sku}", desktop_msg.strip())
111
112     html = _html_wrap(
113         title=f"Approval needed – {action_label}",
114         colour="#d97706",
115         rows=[
116             ("Item", sku),
117             ("Supplier", supplier),
118             ("Total", f"€{total:,.2f}" if total else "-"),
119             ("Betsy's reasoning", reason or "-"),
120             ("Action required", action_label),
121         ],
122     )
123     _email(f"Approval needed – {sku}", html)
124
125
126 def notify_auto_approved(po: dict) -> None:
127     """Fire when Betsy autonomously creates a PO without human intervention."""
128     if not config.NOTIFY_ON_AUTO_PO:
129         return
130
131     po_id = po.get("po_id", "")
132     sku = po.get("sku_id", "")
133     supplier = po.get("supplier_id", "")
134     total = po.get("total_amount") or 0
135
136     _desktop(
137         f"Betsy auto-ordered – {sku}",
138         f"PO {po_id} placed with {supplier}\nTotal: €{total:,.0f}",
139     )
140     html = _html_wrap(
141         title="Betsy placed an order automatically",
142         colour="#7c3aed",
143         rows=[
144             ("PO ID", po_id),
145             ("Item", sku),
146             ("Supplier", supplier),
147             ("Total", f"€{total:,.2f}"),
148             ("Status", "Auto-approved – no action needed"),
149         ],
150     )
151     _email(f"Auto-approved PO – {sku}", html)
152
153
154 def notify_score_drop(supplier_name: str, old_score: float, new_score: float, po_id: str) -> None:
155     """Fire when a supplier's EMA reliability score falls below the threshold."""
156     if not config.NOTIFY_ON_SCORE_DROP:
157         return
158
159     msg = (
160         f"{supplier_name} score dropped: {old_score:.2f} → {new_score:.2f}\n"
161         f"Below warning threshold ({config.SCORE_DROP_THRESHOLD:.2f}). "
162         f"Late delivery on PO {po_id}."
163     )
164     _desktop(f"Supplier alert – {supplier_name}", msg)
165
166     html = _html_wrap(
167         title=f"Supplier reliability warning – {supplier_name}",
168         colour="#dc2626",
169         rows=[
170             ("Supplier", supplier_name),
171             ("Old score", f"{old_score:.4f}"),
172             ("New score", f"{new_score:.4f} ↓"),
173             ("Threshold", f"{config.SCORE_DROP_THRESHOLD:.2f}"),
174             ("Triggered by", f"Late delivery on PO {po_id}"),
175             ("What this means", "Betsy will rank this supplier lower on future orders"),
176         ],
177     )
178     _email(f"Supplier score warning – {supplier_name}", html)
179
180
181 def notify_duplicate_invoice(invoice_1: str, invoice_2: str,
182                             supplier_id: str, amount: float, days_apart: int) -> None:
183     """Fire when a duplicate invoice pair is flagged – re-uses the approval trigger."""
184     if not config.NOTIFY_ON_DUPLICATE:
185         return
186
187     msg = (
188         f"Possible duplicate invoice from {supplier_id}\n"
189         f"{invoice_1} & {invoice_2} – €{amount:,.2f} – {days_apart} days apart"
190     )
191     _desktop("Betsy flagged a duplicate invoice", msg)
192
193     html = _html_wrap(
194         title="Duplicate invoice detected",
195         colour="#dc2626",
196         rows=[
197             ("Supplier", supplier_id),
198             ("Invoice 1", invoice_1),
199             ("Invoice 2", invoice_2),
200             ("Amount each", f"€{amount:,.2f}"),
201             ("Days apart", str(days_apart)),
202             ("Next step", "Review both invoices and approve or decline in Betsy"),
203         ],
204     )
205     _email(f"Duplicate invoice – {supplier_id}", html)
```

What it shows: Fire-and-forget dispatch: desktop via plyer (silent if missing), email on a background thread, every send wrapped so a failure can't break a run.

Source: server/notifier.py:17-51, 86-205 · image rendered by build_snippets.py